Simple Java Assignment: Implementing Methods for Classes and Objects

This Assignment uses code from assignment #3, pt.1 (Encapsulation & Constructors)

Part 1: Creating methods in classes

Objective:
Give each of the Car, Dog, and Student classes three behaviors/methods. Also add and remove variables to make each function work respectively.

Tasks

1. **Car Methods:**.
    - Rev engine: Should print out "VROOM" (or something to that effect).
    - Ride: Should take in a number of miles driven and add that to a running counter. Should also take in speed and return how many minutes said miles would take going a certain # of mph. Should also call Rev Engine.
    - Check Oil: Should return true if Car needs an oil change. Assume Car needs an oil change after it's driven 5k miles since past oil change.
    - Change Oil: Should reset oil change to false.

2. **Dog Methods:**
    - Bark: Should print out "Bark" (or something to that effect).
    - Run: Should print out "Dog is running" every other time, and print out dog is tired otherwise.

3. **Student Methods:**
    - Calculate: Should take two numbers and an operator, and return the answer of an equation using said numbers and operator.

Your Task: The AssignmentTest Class

1. **Create a separate class** and name it AssignmentTest.

2. **Create a main method** within AssignmentTest.

3. **Inside the main method, create an instance of each of your three new classes.**
   Car myCar = new Car();
   Dog myDog = new Dog();
   Student myStudent = new Student();

4. **Test all functionality of classes**

Car.java (Part 1):

```java
// Car.java
public class Car {
    public String make;
    public int year;
    // New variables to track mileage and oil changes
    private int milesDriven = 0;
    private int milesSinceOilChange = 0;

    public Car() {
        // Default constructor from previous assignment.
    }

    // Method to simulate the engine revving.
    public void revEngine() {
        System.out.println("VROOM!");
    }

    // Method to simulate driving the car.
    public int ride(int miles, int speed) {
        // Add miles driven to the counter.
        this.milesDriven += miles;
        this.milesSinceOilChange += miles;

        // Rev the engine before riding.
        revEngine();

        // Calculate and return the time in minutes.
        // Formula: time = (distance / speed) * 60 minutes
        // We use a double for accurate calculation before casting to int.
        double timeInHours = (double) miles / speed;
        return (int) (timeInHours * 60);
    }

    // Method to check if an oil change is needed.
    public boolean checkOil() {
        return this.milesSinceOilChange >= 5000;
    }
```

```
    // Method to reset the oil change counter.
    public void changeOil() {
        System.out.println("Oil change complete. Miles since last change
reset.");
        this.milesSinceOilChange = 0;
    }
}
```

Dog.java (Part 1):

```
// Dog.java
public class Dog {
    public String name;
    public int age;
    // New variable to track the number of runs.
    private int runCount = 0;

    public Dog() {
        // Default constructor from previous assignment.
    }

    // Method to make the dog bark.
    public void bark() {
        System.out.println("Bark!");
    }

    // Method to simulate the dog running.
    public void run() {
        this.runCount++;
        if (this.runCount % 2 == 0) {
            System.out.println("Dog is running!");
        } else {
            System.out.println("Dog is tired.");
        }
    }
}
```

Student.java (Part 1):

```
// Student.java
public class Student {
    public String firstName;
    public String lastName;
    public int studentId;

    public Student() {
        // Default constructor from previous assignment.
    }

    // Method to perform a simple calculation.
    public double calculate(double num1, double num2, char operator) {
        switch (operator) {
            case '+':
                return num1 + num2;
            case '-':
                return num1 - num2;
            case '*':
                return num1 * num2;
            case '/':
                if (num2 != 0) {
                    return num1 / num2;
                } else {
                    System.out.println("Error: Division by zero is not
allowed.");
                    return 0; // Return 0 or throw an exception.
                }
            default:
                System.out.println("Error: Invalid operator.");
                return 0;
        }
    }
}
```

AssignmentTest.java (Part 1):

```
// AssignmentTest.java
public class AssignmentTest {
    public static void main(String[] args) {
```

```
        // Create an instance of each class.
        Car myCar = new Car();
        Dog myDog = new Dog();
        Student myStudent = new Student();

        System.out.println("--- Testing Car Methods ---");
        // Test the Car methods.
        System.out.println("Time to drive 100 miles at 50 mph: " +
myCar.ride(100, 50) + " minutes.");
        System.out.println("Needs oil change? " + myCar.checkOil());
        System.out.println("Time to drive 5000 miles at 50 mph: " +
myCar.ride(5000, 50) + " minutes.");
        System.out.println("Needs oil change? " + myCar.checkOil());
        myCar.changeOil();
        System.out.println("Needs oil change? " + myCar.checkOil());
        System.out.println();

        System.out.println("--- Testing Dog Methods ---");
        // Test the Dog methods.
        myDog.bark();
        myDog.run();
        myDog.run();
        myDog.run();
        System.out.println();

        System.out.println("--- Testing Student Methods ---");
        // Test the Student methods.
        System.out.println("5 + 3 = " + myStudent.calculate(5, 3, '+'));
        System.out.println("10 - 2 = " + myStudent.calculate(10, 2, '-'));
        System.out.println("4 * 6 = " + myStudent.calculate(4, 6, '*'));
        System.out.println("100 / 5 = " + myStudent.calculate(100, 5, '/'));
        System.out.println();
    }
}
```

Part 2: Overloading

In this section, you will take the classes and objects you created in Part 1 and assign new values to their public variables.

Your Task: Modify All classes

1. **Overload ctors such that all inputs can be put in any order. Also add a default for which nothing was input.**
2. **Overload the following methods**
    - Car - Ride: Should allow any order of inputs, and should also have an option to not put in speed (just don't calculate time to drive)
    - Dog - Bark: Should allow Usr to input alternative sound for Dog to make, if nothing is input then Dog should bark.
    - Student - Calculate: Should allow Usr to input equation as either two numbers and an operator, OR a string with the full equation, in the format
      "Num1 + Num2" (One Space in between each token)

Car.java (Part 2):

```
// Car.java
public class Car {
    public String make;
    public int year;
    // Variables for method functionality
    private int milesDriven = 0;
    private int milesSinceOilChange = 0;

    // Overloaded constructors
    // Default constructor
    public Car() {
        this("Unknown", 0);
    }
    // Constructor with make and year
    public Car(String make, int year) {
        this.make = make;
        this.year = year;
    }
    // Constructor with make only
    public Car(String make) {
        this(make, 0);
    }
    // Constructor with year only
    public Car(int year) {
        this("Unknown", year);
    }
```

```
    // Method to simulate the engine revving.
    public void revEngine() {
        System.out.println("VROOM!");
    }

    // Overloaded ride method (with speed)
    public int ride(int miles, int speed) {
        this.milesDriven += miles;
        this.milesSinceOilChange += miles;
        revEngine();
        double timeInHours = (double) miles / speed;
        return (int) (timeInHours * 60);
    }

    // Overloaded ride method (without speed)
    public void ride(int miles) {
        this.milesDriven += miles;
        this.milesSinceOilChange += miles;
        revEngine();
        System.out.println("Drove " + miles + " miles. Time not calculated.");
    }

    public boolean checkOil() {
        return this.milesSinceOilChange >= 5000;
    }

    public void changeOil() {
        System.out.println("Oil change complete. Miles since last change
reset.");
        this.milesSinceOilChange = 0;
    }
}
```

Dog.java (Part 2):

```
// Dog.java
public class Dog {
    public String name;
    public int age;
    private int runCount = 0;
```

```
    // Overloaded constructors
    // Default constructor
    public Dog() {
        this("Unknown", 0);
    }
    // Constructor with name and age
    public Dog(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // Constructor with name only
    public Dog(String name) {
        this(name, 0);
    }
    // Constructor with age only
    public Dog(int age) {
        this("Unknown", age);
    }

    // Overloaded bark method (default sound)
    public void bark() {
        System.out.println("Bark!");
    }

    // Overloaded bark method (user-defined sound)
    public void bark(String sound) {
        System.out.println(sound + "!");
    }

    public void run() {
        this.runCount++;
        if (this.runCount % 2 == 0) {
            System.out.println("Dog is running!");
        } else {
            System.out.println("Dog is tired.");
        }
    }
}
```

Student.java (Part 2):

```java
// Student.java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Student {
    public String firstName;
    public String lastName;
    public int studentId;

    // Overloaded constructors
    // Default constructor
    public Student() {
        this("Unknown", "Unknown", 0);
    }
    // Constructor with all details
    public Student(String firstName, String lastName, int studentId) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.studentId = studentId;
    }
    // Constructor with first and last name
    public Student(String firstName, String lastName) {
        this(firstName, lastName, 0);
    }
    // Constructor with student ID only
    public Student(int studentId) {
        this("Unknown", "Unknown", studentId);
    }

    // Overloaded calculate method (two numbers and a character operator)
    public double calculate(double num1, double num2, char operator) {
        switch (operator) {
            case '+':
                return num1 + num2;
            case '-':
                return num1 - num2;
            case '*':
```

```
                return num1 * num2;
            case '/':
                if (num2 != 0) {
                    return num1 / num2;
                } else {
                    System.out.println("Error: Division by zero is not
allowed.");
                    return 0;
                }
            default:
                System.out.println("Error: Invalid operator.");
                return 0;
        }
    }

    // Overloaded calculate method (equation as a string)
    public double calculate(String equation) {
        // Use a regular expression to parse the equation string
        Pattern pattern = Pattern.compile("(\\d+)\\s*([+\\-*/])\\s*(\\d+)");
        Matcher matcher = pattern.matcher(equation);

        if (matcher.find()) {
            double num1 = Double.parseDouble(matcher.group(1));
            char operator = matcher.group(2).charAt(0);
            double num2 = Double.parseDouble(matcher.group(3));
            return calculate(num1, num2, operator);
        } else {
            System.out.println("Error: Invalid equation format. Please use
'Num1 Op Num2'.");
            return 0;
        }
    }
}
```

AssignmentTest (Part 2):

```
// Student.java
public class AssignmentTest {
    public static void main(String[] args) {
        System.out.println("--- Testing Overloaded Constructors and Methods
```

```
---");

        // Overloaded Car Constructors and Methods
        Car car1 = new Car("Honda", 2022);
        Car car2 = new Car(2020);
        car1.revEngine();
        System.out.println("Time to drive 100 miles at 50 mph: " +
car1.ride(100, 50) + " minutes.");
        car2.ride(50); // Calling the overloaded ride method without speed
        System.out.println();

        // Overloaded Dog Constructors and Methods
        Dog dog1 = new Dog("Buddy", 5);
        Dog dog2 = new Dog("Max");
        dog1.bark(); // Calling default bark
        dog2.bark("WOOF!"); // Calling overloaded bark with a custom sound
        System.out.println();

        // Overloaded Student Constructors and Methods
        Student student1 = new Student("Jane", "Doe", 12345);
        Student student2 = new Student(98765);
        System.out.println("10 - 2 = " + student1.calculate(10, 2, '-')); //
Original calculate method
        System.out.println("15 + 7 = " + student2.calculate("15 + 7")); //
Overloaded calculate method with a string
        System.out.println();
    }
}
```